

JVM – Máquina Virtual Java

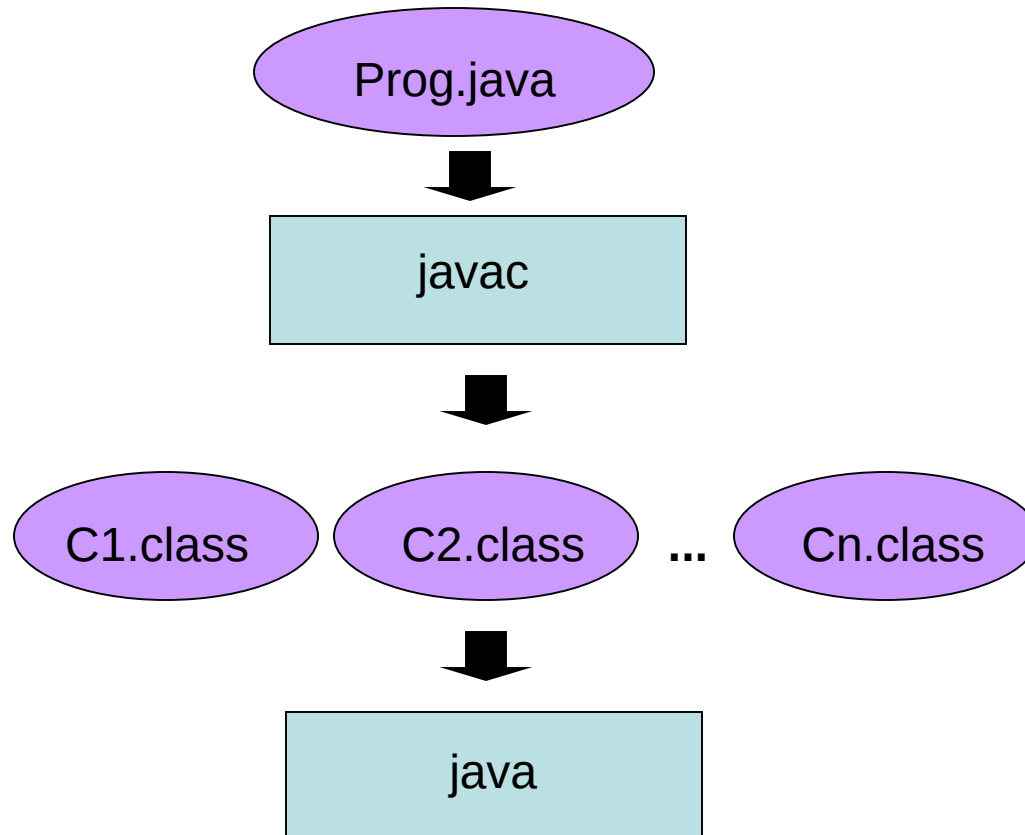
Cristiano Damiani Vasconcellos

cristiano.vasconcello@udesc.br

JVM - Introdução

- Processador virtual;
- Possui seu próprio conjunto de instruções;
- Arquitetura baseada em pilha de operandos;
- Responsável pela tradução e otimização do código, gerenciamento de memória (incluindo o coletor de lixo), tratamento de exceções e gerenciamento das *threads*.

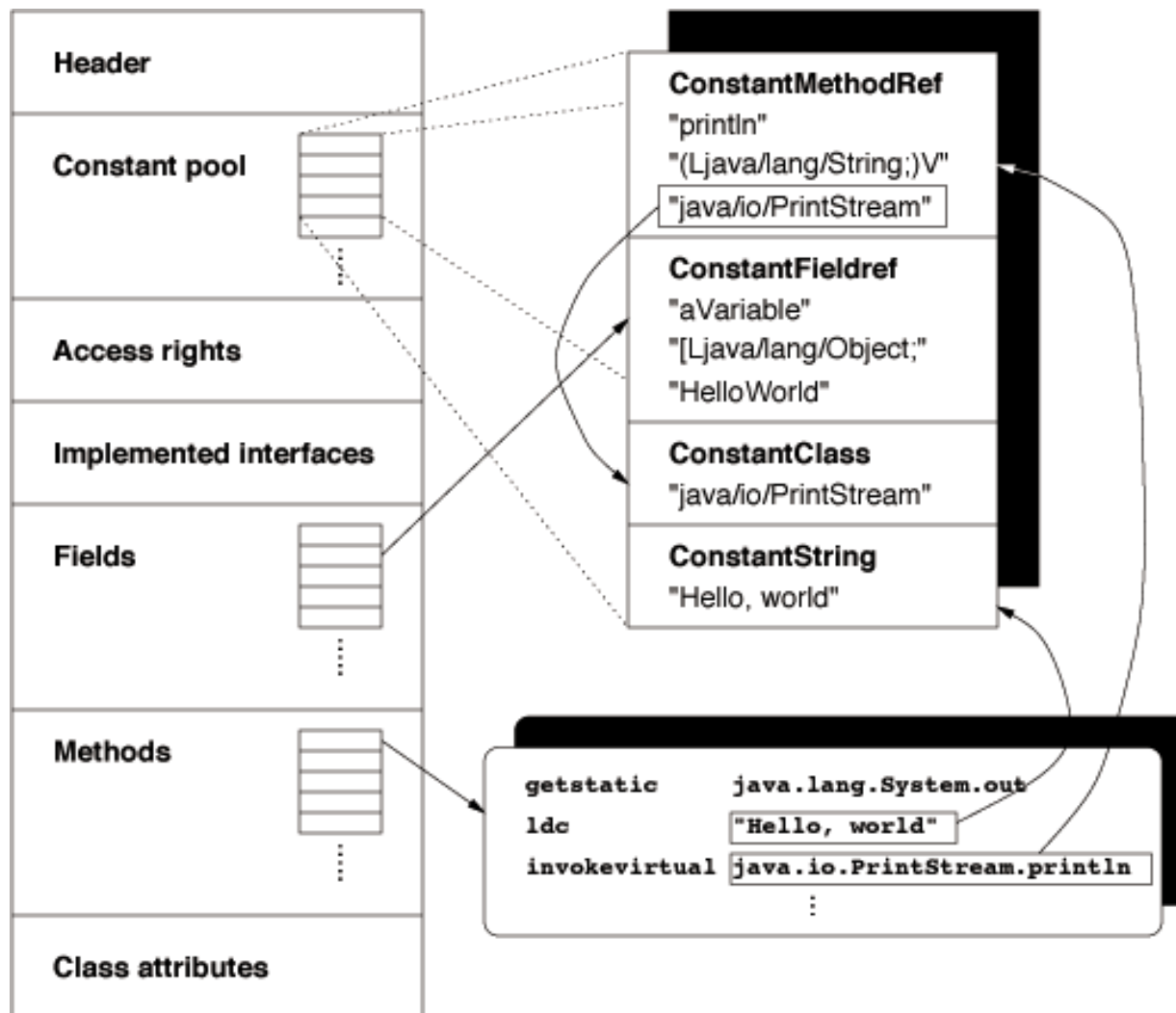
JVM - Introdução



Arquivos .class

- Independente de plataforma;
- Representa uma classe ou interface;
- Executado pela JVM.

Formato do arquivo .class



Formato do arquivo. class

- Header : Número mágico, maior e menor versão.
- Constante pool : *Strings*, nomes de classes, nomes de interfaces, nomes de atributos, constantes numéricas maiores que dois bytes.
- Access rights : public, final, interface, abstract, super.
- Fields : Atributos (campos da classe).
- Methods : Métodos.
- Atributes : Alguns atributos do arquivo, como por exemplo o nome do arquivo fonte.

Tipos Básicos

- byte – 8 bits.
- short – 16 bits.
- int – 32 bits.
- long – 64 bits.
- char – 16 bits (Unicode characters).
- float – 32 bits.
- double – 64 bits.
- boolean (em geral é armazenado como um inteiro).
- returnAddress – ponteiros para *opcodes*.
- Referências – classes, arranjos, interfaces e *null*.

Organização da Memória

Pilhas - Cada *thread* possui sua pilha de execução que funciona de forma similar a pilha de outras linguagens de programação.

Heap - Armazena instâncias de classes e arranjos.

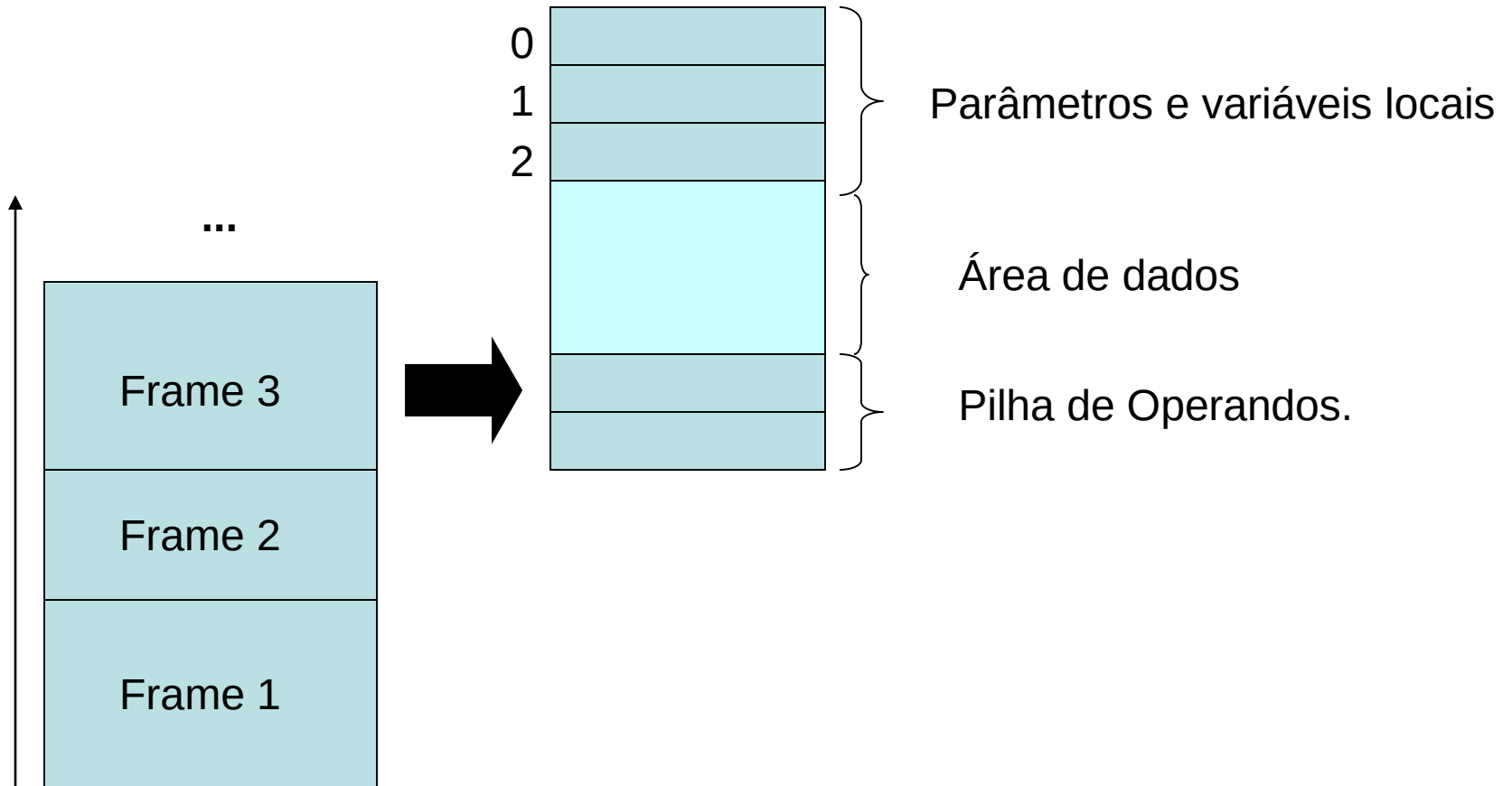
Runtime Constant Pool – Uma por classe ou interface, contém a tabela de constantes armazenada no arquivo .class.

Área de métodos – Similar ao segmento de texto.

Organização da Memória

Frames – Pode ser visto como uma abstração para o registro de ativação dos métodos, os parâmetros e as variáveis locais são acessados através de um índice relativo ao *frame*. Cada posição indexada no *frame* possui 32 bits (alinhamento por palavra). Inclui também a pilha de operandos e uma área de dados.

Frames



Pilha de execução (cada *thread* possui a sua)

Frames

- O tamanho do frame é calculado em tempo de compilação.
- Os parâmetros e as variáveis locais são acessados através de um índice. Em métodos de instância o índice zero, primeiro parâmetro, contém uma referência ao objeto para o qual o método foi chamado (*this*).
- A área de dados contém uma referência para a área de constantes, informações para o retorno do método e informações para o tratamento de exceções.

Instruções

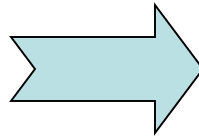
- *Opcode* com apenas 1 byte.
- Algumas instruções possuem 1 ou mais parâmetros.
- A maioria das instruções opera sobre um tipo específico, nesses casos, a primeira letra no mnemônico da instrução informa esse tipo.

Algumas instruções p/ inteiros

- *iconst_n* – Empilha uma constante (0,1,2,3,4 e 5) na pilha de operandos (e.g. *iconst_0*).
- *iload* <parâmetro> – Empilha o valor inteiro armazenado em uma variável local na pilha de operandos. O parâmetro para *iload* é um índice do frame corrente (e.g. *iload 4*).
- *istore* <parâmetro> – Desempilha o valor armazenado no topo da pilha de operandos, armazenando esse valor em uma variável local. O parâmetro para *istore* é um índice do frame corrente (e.g. *istore 4*).
- *iadd* – Desempilha os 2 operando no topo da pilha executa a soma e empilha o resultado.
- *bipush* <parâmetro> – Empilha o parâmetro (*byte*) como um *int* (e.g. *bipush 27*).

Exemplo

```
int a, b, c;  
a = 1;  
b = 20;  
c = b + a * 2;
```



```
iconst_1  
istore 1  
bipush 20  
istore 2  
iload 2  
iload 1  
iconst_2  
imul  
iadd  
istore 3
```

Esquema de Tradução

$S \rightarrow id = E \{gerar(istore, id.posLexval)\}$

$E \rightarrow E + T \{gerar(iadd)\}$

$E \rightarrow T$

$T \rightarrow T * F \{gerar(imul)\}$

$T \rightarrow F$

$F \rightarrow const \{gerar(bipush, const.lexval)\}$

$F \rightarrow id \{gerar(ilogd, id.posLexval)\}$

$F \rightarrow (E)$

Algumas instruções p/ inteiros

if_icmp<cond> <label> - Desvia a execução para o label se o resultado da comparação dos dois operandos no topo da pilha resultar em um valor verdade.

- *eq* *valor1 = valor2*
- *ne* *valor1 ≠ valor2*
- *lt* *valor1 < valor2*
- *le* *valor1 ≤ valor2*
- *gt* *valor1 > valor2*
- *ge* *valor1 ≥ valor2*

(e.g. if_icompeq L10)

Algumas instruções

goto <label> - Desvio incondicional.

invokestatic <método> - Chama um método de classe.

invokevirtual <método> - Chama um método de instância.

ldc <índice> - Empilha um item da *runtime constant pool*.

$B \rightarrow B_1 \text{ or } M \ C$	<pre>{corrigir(B1.listaf, M.label); B.listav = merge(B1.listav, C.listav); B.listaf = C.listaf;}</pre>
$B \rightarrow B_1 \text{ and } M \ C$	<pre>{corrigir(B1.listav, M.label); B.listaf = merge(B1.listaf, C.listaf); B.listav = C.listav;}</pre>
$B \rightarrow C$	<pre>{B.listav = C.listav; B.listaf = C.listaf;</pre>
$C \rightarrow \text{not } C$	<pre> C.listav = C.listaf; C.Listaf = C.listav;</pre>
$C \rightarrow (B)$	<pre>{C.listav = B.listav; C.Listaf = B.listaf;}</pre>
$C \rightarrow E_1 \text{ rel } E_2$	<pre> C.listav = criaLista(proxInst); C.listaf = criaLista(proxInst + 1); gerar(if_icmp[rel] goto ____); gerar(goto ____); }</pre>
$M \rightarrow e$	<pre>{M.label = novolabel();}</pre>

Esquema de Tradução

$S \rightarrow \text{if } (B) \ M \ S$	$\{\text{corrigir}(B.\text{listav}, M.\text{label});$ $\quad \text{corrigir}(B.\text{listaf}, \text{ novoLabel}()); \}$
$S \rightarrow \text{if}(B) \ M_1 \ S \ N \ \text{else} \ M_2 \ S$	$\{\text{corrigir}(B.\text{listav}, M_1.\text{label});$ $\quad \text{corrigir}(B.\text{listaf}, M_2.\text{label});$ $\quad \text{corrigir}(N.\text{listav}, \text{ novoLabel}()); \}$
$S \rightarrow \text{while} \ M_1 \ (B) \ M_2 \ S$	$\{\text{corrigir}(B.\text{listav}, M_2.\text{label});$ $\quad \text{gerar}(\text{goto } M_1.\text{label})$ $\quad \text{corrigir}(B.\text{listaf}, \text{ novoLabel}()); \}$
$N \rightarrow \epsilon$	$\{N.\text{listav} = \text{criarLista}(\text{ProxInstr});$ $\quad \text{gerar}(\text{goto } __);$

Descritores de Tipos

- Representam os tipos de atributos e métodos em um arquivo *.class*;
- Usados no acesso a atributos e chamadas de métodos.

Exemplos:

[Ljava/lang/String; representa *java.lang.String[]*

[[I representa *int[][]*

Descritores de Tipos

Tipos básicos:

- Z – boolean;
- B – Byte;
- C – char;
- S – short;
- I – int;
- J – long;
- F – float;
- D – double;
- V – void.

Descritores de Tipos

Um descritor de classe representa o nome de uma classe ou interface, por exemplo:

java.lang.String é representado por:

java/lang/String

Obs: Quando um descritor de classe ocorre na formação de um descritor de tipo, seu início é marcado com um “L” e seu final marcado com um “;”.

Um arranjo é representado por [.

Descritores de Métodos

(<Descritor de Tipo>*)<Descritor de Tipo>

Exemplos:

O tipo do método `int foo(int a, String s)`
é representado por: `(Ljava/lang/String;)I`

O tipo do método `void bar(int a, double b[])`
é representado por: `(I[D)V`

Jasmin

Montador para a JVM.

Uso:

```
java -jar jasmin MeuPrograma.j
```


Exemplos Jasmin

```
.class public Alo
.super java/lang/Object

.method public <init>()V
    aload_0
    invokevirtual java/lang/Object/<init>()V
    return
.end method

.method public static main([Ljava/lang/String;)V
    .limit stack 2
    .limit locals 1

    getstatic java/lang/System/out Ljava/io/PrintStream;
    ldc "Alo"
    invokevirtual java/io/PrintStream/println(Ljava/lang/String;)V
    return
.end method
```

Exemplos Jasmin

```
public static void main(String args[])
{
    int b, a = 1;

    while (a < 10)
    {
        b = a * 2;
        System.out.println(b);
        a++;
    }
}
```

Exemplos Jasmin

```
.method public static main([Ljava/lang/String;)V
    .limit stack 2
    .limit locals 4
    iconst_1
    istore_2
l1:
    iload_2
    bipush 10
    if_icmpge l2
    iload_2
    iconst_2
    imul
    istore_1
    getstatic java/lang/System/out Ljava/io/PrintStream;
    iload_1
    invokevirtual java/io/PrintStream/println(I)V
    iinc 2 1
    goto l1
l2:
    return
.end method
```

Referências

Java™ Virtual Machine Specification (2nd Edition).
Tim Lindholm and Frank Yellin.

Jasmin User Guide. Jonathan Meyer.